

Project Specific Expressions from Nixpkgs for Sphinx documentation

Rohit Goswami · 2020-12-22 · documentation · tools · nix · workflow

Short post on making minimal changes to derivations in `nixpkgs` at a project level using `callPackage()` along with GH-Actions for deployment of `sphinx` documentation.

Background

As part of my work on the [Symengine documentation](#)^[1], I had originally thought of using `nix` for reproducible builds for each of the language bindings with GH-Actions. There exists a derivation in the upstream package repository, but it was outdated (`v6.0.0` instead of `v6.0.1`)^[2]. Normally this would simply require a PR to be fixed; but since documenting different language bindings requires specific flags and tests were largely not an issue, I needed to make a project level derivation for each repository.

Project Layout

The standard `niv` setup, as described in the [longer tutorial post](#) will suffice.

```
niv init -b master
```

Since the bindings were for `python`, `mach-nix` was leveraged for the `shell.nix`; reasons for which [have been outlined elsewhere](#). This led to the following expression:

```
``nix { hLines=["4"] } let sources = import ./nix/sources.nix; pkgs = import sources.nixpkgs { }; nsymengine = pkgs.callPackage ./nix/nsymengine.nix { }; mach-nix = import (builtins.fetchGit { url = "https://github.com/DavHau/mach-nix/"; ref = "refs/tags/3.1.1"; }) { pkgs = pkgs; }; customPython = mach-nix.mkPython rec { requirements = builtins.readFile ./requirements.txt; }; in pkgs.mkShell { buildInputs = with pkgs; [ customPython cmake nsymengine bashInteractive sage which ]; }
```

The highlighted line calls ``nsymengine.nix`` which is derived from the upstream expression. Since ``callPackage`` is being used from the ``niv`` pinned sources, the expression is completely project-specific and reproducible.

```
### Upstream {#upstream}
```

For completion; the upstream expression is reproduced. We will actually grab this directly with ``show-derivation``:

```
```bash
nix show-derivation -f "<nixpkgs>" symengine > nix/nsymengine.nix
```

Which gives us:

```

{ lib, stdenv
, fetchFromGitHub
, cmake
, gmp
, flint
, mpfr
, libmpc
}:

stdenv.mkDerivation rec {
 pname = "symengine";
 version = "0.6.0";

 src = fetchFromGitHub {
 owner = "symengine";
 repo = "symengine";
 rev = "v${version}";
 sha256 = "129iv9maabmb42ylfdv0l0g94mcbf3y4q3np175008rcqdr8z6h1";
 };

 nativeBuildInputs = [cmake];

 buildInputs = [gmp flint mpfr libmpc];

 cmakeFlags = [
 "-DWITH_FLINT=ON"
 "-DINTEGER_CLASS=flint"
 "-DWITH_SYMENGINE_THREAD_SAFE=yes"
 "-DWITH_MPC=yes"
 "-DBUILD_FOR_DISTRIBUTION=yes"
];

 doCheck = true;

 checkPhase = ''
 ctest
 '';

 meta = with lib; {
 description = "A fast symbolic manipulation library";
 homepage = "https://github.com/symengine/symengine";
 platforms = platforms.unix ++ platforms.windows;
 license = licenses.bsd3;
 maintainers = [maintainers.costrouc];
 };
}

```

## Repo-native

The truncated expression meant to be bundled with the repo is reproduced below.

```

{ lib, stdenv
, fetchFromGitHub
, cmake
, gmp
, flint
, mpfr
, libmpc
}:

stdenv.mkDerivation rec {
 pname = "symengine";
 name = "symengine";
 version = "44eb47e3bbfa7e06718f2f65f3f41a0a9d133b70"; # From symengine-version.txt

 src = fetchFromGitHub {
 owner = "symengine";
 repo = "symengine";
 rev = "${version}";
 sha256 = "137cxk3x8vmr4p5x0knzjplir0slw0gmwhzi277si944i33781hd";
 };

 nativeBuildInputs = [cmake];

 buildInputs = [gmp flint mpfr libmpc];

 cmakeFlags = [
 "-DWITH_FLINT=ON"
 "-DINTEGER_CLASS=flint"
 "-DWITH_SYMENGINE_THREAD_SAFE=yes"
 "-DWITH_MPC=yes"
 "-DBUILD_TESTS=no"
 "-DBUILD_FOR_DISTRIBUTION=yes"
];

 doCheck = false;
}

Derived from the upstream expression : https://github.com/r-
ryantm/nixpkgs/blob/34730e0640710636b15338f20836165f29b3df86/pkgs/development/libraries/symengine/default.nix

```

Some project-specific notes:

- `m2r2` was used to inject the project `README.md` into the `Sphinx reStructuredText` index file
- To prevent `symengine` from picking up the system `python`, a pure shell was required `nix-shell --pure --run bash`
- `symengine-version.txt` was already being used by the CI setup for cloning the right commit, and though a text file for hash storage is inelegant, it is a valid approach without `nix`

## GH-Actions and Deployment

Having obtained an environment; the penultimate step involved a trivial shell script which uses the fantastic `nix-shell`:

```

#!/usr/bin/env nix-shell
#!nix-shell ../shell.nix -i bash

python setup.py build_ext --inplace
sphinx-build docs/ genDocs/

```

Finally an action definition in `.github/workflows/mkdocs.yaml`:

```
on:
 push:
 branches:
 - main
 - master

name: Make Sphinx API Docs

jobs:
 mkdocs:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v2

 - uses: cachix/install-nix-action@v12
 with:
 nix_path: nixpkgs=channel:nixos-unstable

 - name: Build
 run: ./bin/docbuilder.sh

 - name: Deploy
 uses: peaceiris/actions-gh-pages@v3
 with:
 github_token: ${ secrets.GITHUB_TOKEN }
 publish_dir: ./genDocs
```

## Conclusion

In the end, to ensure easy long-term maintenance by the other maintainers, it was decided that the `nix` derivations should be trashed in favor of `conda` madness. Though most changes should be contributed upstream, the efficient re-use of upstream expressions is still viable for certain projects, especially those related to documentation.